

Trabajo Práctico – Desarrollo Frontend con Lit y Tailwind

Fecha límite: 30 de septiembre. Debe enviarse un email con el link al repo a fgonzalez@untdf.edu.ar con copia a nsartori@untdf.edu.ar

Modalidad

Grupal: 2 o 3 integrantes.

Repositorio: proyecto publicado en un repo público en GitHub.

Commits: todos los miembros del grupo deben realizar commits (se evaluará la participación).

El repo debe tener un archivo README.md con la documentación y los integrantes del grupo.

Objetivo

Desarrollar el frontend de un e-commerce (puede ser supermercado, librería, tienda de ropa, etc.) utilizando Lit para los componentes web y TailwindCSS para los estilos.

Los datos deberán consumirse desde la API pública disponible en

<https://ecommerce.fedegonzalez.com/docs>

Requisitos Generales

Uso de Lit para crear los componentes.

Uso de TailwindCSS (no bootstrap) para el diseño.

Prioridad mobile-first para el diseño.

Estructura de la aplicación basada en tres archivos HTML independientes:

- index.html

- listado.html?categoria=xxxx

- ficha.html?producto=xxxx

Ejemplo de cómo obtener parámetros desde la URL en JavaScript:

```
const params = new URLSearchParams(window.location.search);  
const categoria = params.get("categoria");  
console.log(categoria); // Si la URL es listado.html?categoria=5, esto mostrará "5"
```

Nota: para armar la listado.html, reciben el id de la categoría en la url, traen la lista de productos con el endpoint "products" y filtran con javascript los datos para solo mostrar la categoría de interés. Para armar ficha.html reciben el id del producto en la url y consultan a ese producto en la api con products/id

Pantallas a Desarrollar

1. Home

Debe mostrar productos de diferentes categorías. Se pueden usar etiquetas (ejemplo: destacado, promoción, mostrar_en_home) para decidir qué productos mostrar.

2. Categoría

Muestra los productos correspondientes a una categoría seleccionada por ejemplo listado.html?categoria=xxxx

El diseño puede ser en forma de grid o lista. Puede haber productos destacados (opcional).

3. Ficha de producto

Muestra información detallada del producto seleccionado ficha.html?producto=xxxx

4. Carrito de compras

Implementado con localStorage (setItem, getItem).

Debe persistir la información aunque se recargue la página.

Debe mostrar:

Cantidad total de productos.

Precio total.

Debe ofrecer estas opciones:

Vaciar el carrito.

Agregar/Quitar productos desde el home, el listado o su ficha.

El carrito puede ser sencillo (solo cantidad y precio total) o más completo (listado detallado de productos). Los alumnos deben investigar cómo utilizar localStorage.

Consideraciones Técnicas

Se trabajará solo con verbos GET de la API. No es necesario implementar POST, PUT o DELETE.

TypeScript es opcional.

Diseño y experiencia de usuario serán valorados.